



Yesterday 12:52 PM

Continue KOHLER AI Research Lab — Track 3, Phase 4.

Worked for 11s >

Absolutely. We'll continue from **Phase 3 → Phase 4** for **KOHLER AI Research Lab — Track 3: Unified Enterprise AI Agent**.

### Where we are

We have established:

#### Core concept:

A secure **Unified Enterprise AI Agent** that sits above KOHLER's existing agents, enterprise knowledge, data, tools, and workflows.

#### Core lifecycle:

Understand → Decompose → Route → Execute → Verify → Respond

Phase 3 focused heavily on **orchestration**, including dynamic routing, task decomposition, parallel/sequential execution, shared state, failure recovery, and verification.

## PHASE 4 — SYSTEM ARCHITECTURE & TECHNICAL DESIGN

The goal of Phase 4 is to convert the research into a **concrete technical architecture** that you can eventually present as your solution.

We'll cover it in areas:

1. Overall System Architecture
2. Master Orchestrator / Agent Brain
3. Specialist Agents
4. Knowledge + RAG Architecture
5. Tools, APIs & Enterprise Systems
6. Memory & Context Management
7. Security, Permissions & Governance

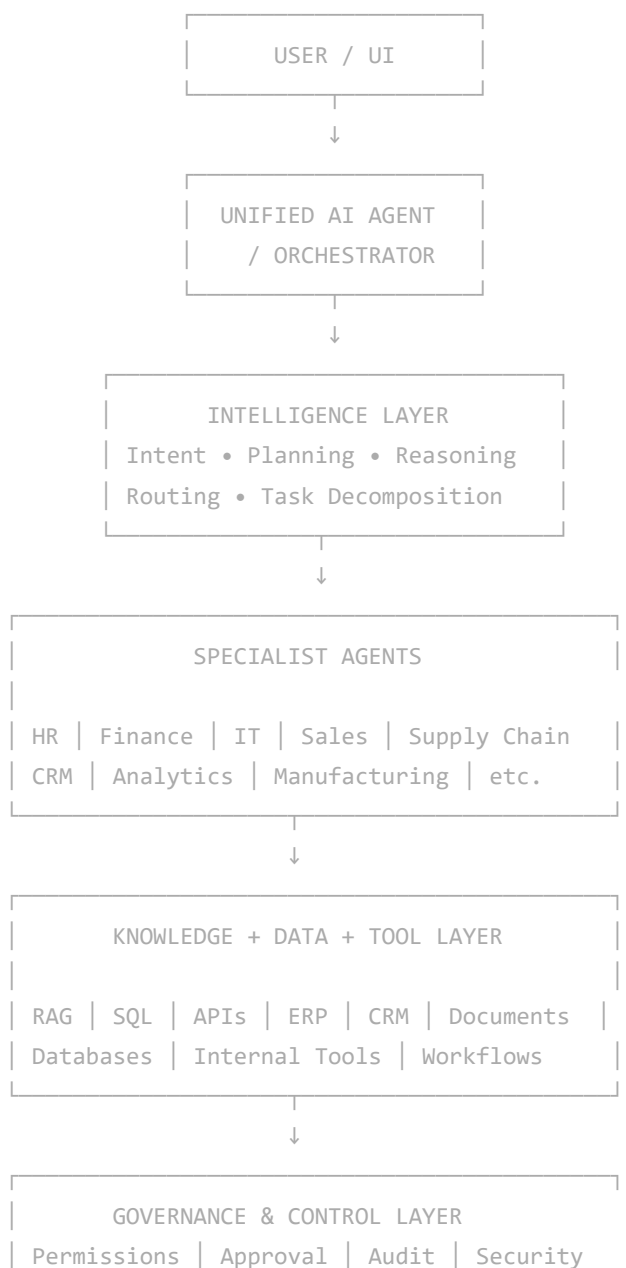
8. Observability & Verification
9. End-to-End Workflow
10. MVP Architecture & Technology Stack

Since you previously wanted the research notes relatively concise, we'll keep each area technical but note-friendly.

## Area 1 — Overall System Architecture

The architecture should be thought of as **layers**, rather than simply "one agent talking to other agents."

### High-level architecture





## Important idea

The **Unified Agent** is not replacing every existing agent.

Instead:

It acts as the enterprise control and coordination layer connecting the user to existing AI capabilities.

For example:

A user asks:

"Why did sales of Product X decline in Europe last quarter?"

The system may:

1. Understand the question.
2. Break it into subtasks.
3. Query sales data.
4. Retrieve relevant product information.
5. Ask the supply-chain agent about availability.
6. Ask the market/CRM agent about customer trends.
7. Combine the results.
8. Verify conflicting information.
9. Produce one grounded answer.

The user doesn't need to know **which agent performed which task**.

## The 6 major architectural components

### 1. User Interface

The single entry point.

Could be:

- Web application
- Enterprise chat interface
- Microsoft Teams/Slack-style interface
- API

## 2. Orchestrator

The **central decision-making layer**.

Responsible for:

- Understanding intent
- Breaking down tasks
- Selecting agents
- Selecting tools
- Managing dependencies
- Maintaining task state
- Handling failures
- Coordinating parallel execution
- Verification

This is the **core of your project**.

## 3. Specialist Agents

Existing or newly developed agents responsible for specific domains.

Examples:

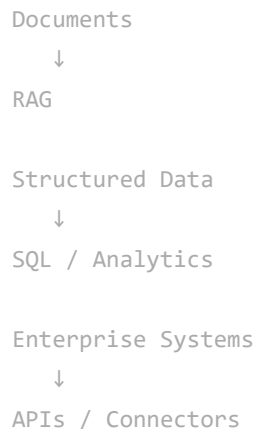
HR Agent  
Finance Agent  
Sales Agent  
IT Agent  
Supply Chain Agent  
Manufacturing Agent  
Analytics Agent

Each agent should have:

- Specific capabilities
- Domain knowledge
- Authorized tools
- Defined inputs/outputs
- Permission boundaries

## 4. Enterprise Knowledge & Data

The system needs access to multiple information sources.



This prevents the LLM from relying purely on its internal knowledge.

## 5. Tools & Actions

The system should not only **answer**.

It should also be capable of **doing things**.

Examples:

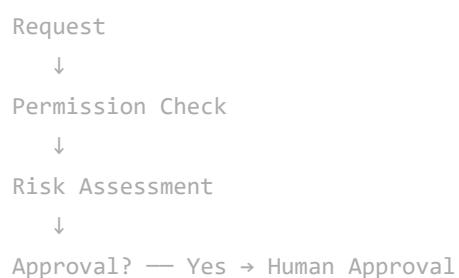
```
Search database
Create ticket
Check inventory
Generate report
Update CRM
Send notification
Create purchase request
```

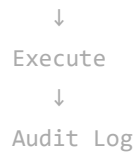
But actions must pass through permission and risk controls.

## 6. Governance Layer

This is particularly important for an enterprise system.

Before an action happens:





This makes the system **enterprise-safe** rather than simply **autonomous**.

## Key architectural principle

The architecture should follow:

**Centralized intelligence + decentralized capabilities + centralized governance**

Meaning:

- **Centralized intelligence:** one orchestrator understands the overall task.
- **Decentralized capabilities:** specialist agents handle their domains.
- **Centralized governance:** security, permissions, verification and auditing remain consistent.

This is one of the strongest architectural ideas for your Track 3 solution.

## Notes for your notebook

### Phase 4 — Area 1: Overall Architecture

**Goal:** Build a unified enterprise control layer over existing agents, knowledge, data and tools.

**Layers:**

1. User Interface
2. Orchestrator
3. Intelligence/Planning
4. Specialist Agents
5. Knowledge/Data/Tools
6. Security & Governance

**Core principle:**

Centralized intelligence + decentralized capabilities + centralized governance.

**Orchestrator responsibilities:**

- Intent understanding

- Task decomposition
- Agent/tool selection
- Execution coordination
- State management
- Failure recovery
- Verification
- Response generation

**Important:**

The system **does not** replace existing agents. It connects and coordinates them through one intelligent enterprise interface.

**Area 1 takeaway**

Your architecture is **not** simply a multi-agent system.

It is:

An enterprise orchestration and control layer that dynamically connects the right agents, data and tools while enforcing permissions, verification and governance.

Next, Area 2 = Master Orchestrator / Agent Brain, where we'll design exactly how your main agent decides what to do, which agent to call, when to parallelize tasks, and when to ask for human approval.

    ...  Sources

explain all areas in short

Sure. Here is **Phase 4 — all areas in short**, focused on what you actually need for the KOHLER Track 3 solution.

## Phase 4 — Technical Architecture

### Area 1 — Overall System Architecture

Design the complete system from **user → orchestrator → specialist agents → tools/data → response**.

**Key idea:** One unified interface coordinates many enterprise capabilities.

## Area 2 — Master Orchestrator / Agent Brain

The orchestrator is the **central controller**.

It:

- Understands user intent
- Breaks complex requests into tasks
- Selects the right agents/tools
- Decides parallel vs sequential execution
- Tracks task state
- Handles failures
- Combines results

**Example:**

"Why are sales declining?"

→ Sales Agent + Finance Agent + Supply Chain Agent → combine findings → answer.

## Area 3 — Specialist Agents

Create/plugin in domain-specific agents.

Examples:

- Finance Agent
- HR Agent
- Sales Agent
- IT Agent
- Supply Chain Agent
- Manufacturing Agent

Each agent has its own:

- Knowledge
- Tools
- Instructions
- Permissions
- Responsibilities

The orchestrator decides **which agent should work on what**.

## Area 4 — Knowledge + RAG Architecture

Agents need access to enterprise information.

Sources can include:

PDFs / Documents



Chunking



Embeddings



Vector DB



RAG



Agent

For structured information:

Database → SQL → Agent

This allows agents to use **company-specific and up-to-date information** instead of relying only on the LLM.

## Area 5 — Tools, APIs & Enterprise Systems

Agents need tools to **take actions**, not just answer questions.

Examples:

- SQL databases
- ERP
- CRM
- Email
- Ticketing systems
- Inventory systems
- Internal APIs
- Document systems

Example:

"Check inventory and create a purchase request."

The agent can retrieve inventory → determine requirement → create request.

## Area 6 — Memory & Context Management

The system needs to remember the **right information for the current task**.

Types:

### Short-term memory

→ Current conversation/task.

### Long-term memory

→ Useful persistent preferences or enterprise context.

### Task memory

→ What different agents have already done.

Example:

```
graph TD
    A[User request] --> B[Orchestrator]
    B --> C["Finance Agent → result"]
    C --> D["Supply Chain Agent → uses Finance result"]
    D --> E[Final answer]
```

This prevents agents from working with disconnected context.

## Area 7 — Security, Permissions & Governance

This is extremely important for an enterprise system.

Every action should be checked against:

- User identity
- Role
- Permissions
- Data access
- Risk level
- Agent permissions

Example:

```
graph TD
    A[User wants to modify Finance document] --> B[ ]
```



This is where your **HITL feature** fits.

For your example:

User inserts new financial data → Finance Agent prepares the modification → Finance Admin receives approval request → Admin approves/rejects → document is updated only if approved.

## Area 8 — Observability & Verification

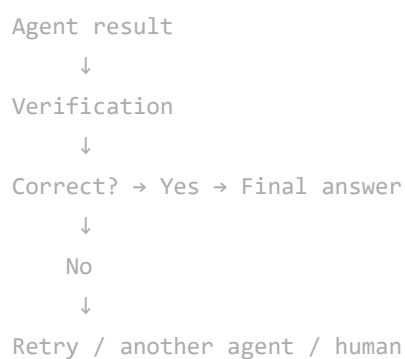
The system should be able to answer:

**"What did the agents do, and why?"**

Track:

- Agent calls
- Tool calls
- Inputs/outputs
- Errors
- Execution time
- Decisions
- Approval history

Also verify outputs before returning them.



## Area 9 — End-to-End Workflow

Your complete workflow can be:



This becomes your **main architecture flow diagram**.

## Area 10 — MVP Architecture & Tech Stack

For your prototype, don't try to build a huge enterprise platform.

A practical MVP could use:

| Component       | Possible Technology   |
|-----------------|-----------------------|
| Frontend        | React / Next.js       |
| Backend         | Python + FastAPI      |
| Agent framework | LangGraph             |
| LLM             | Gemini / OpenAI       |
| RAG             | LangChain + vector DB |

**Component****Possible Technology**

Vector DB

Chroma / FAISS

Database

PostgreSQL

APIs

REST

Authentication

JWT / OAuth

HITL

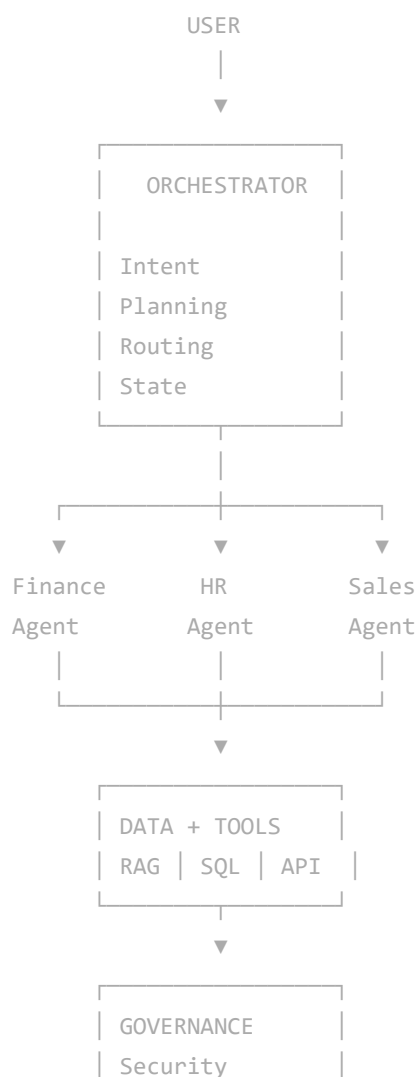
Approval workflow

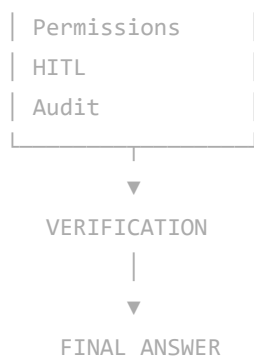
Monitoring

Logs + LangSmith

The important thing is demonstrating the **architecture and workflow**, not building every enterprise integration.

## ★ Phase 4 in one picture





## Most important areas for your prototype

If you're short on time, focus heavily on:

1. **Orchestrator** → how agents are coordinated
2. **Specialist agents** → what capabilities they provide
3. **RAG/data** → how enterprise knowledge is accessed
4. **HITL + permissions** → how risky actions are controlled
5. **Verification** → how you prevent bad outputs
6. **User-provided data** → your evaluator can insert new data and test the system

That last point is especially useful for your demo because you can show:

**User uploads/inserts new data → system processes it → appropriate agent uses it → sensitive modification triggers HITL → authorized admin approves → system updates the enterprise knowledge/data source.**

That gives you a concrete demonstration rather than only showing agents talking to each other.



↳ A practical MVP could use:

i dont want any kind of paid software in my workflow

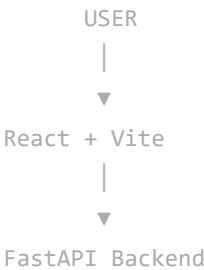
Absolutely. For your **KOHLER Track 3 prototype**, we can design the entire workflow around **free/open-source software** and avoid paid APIs or subscriptions.

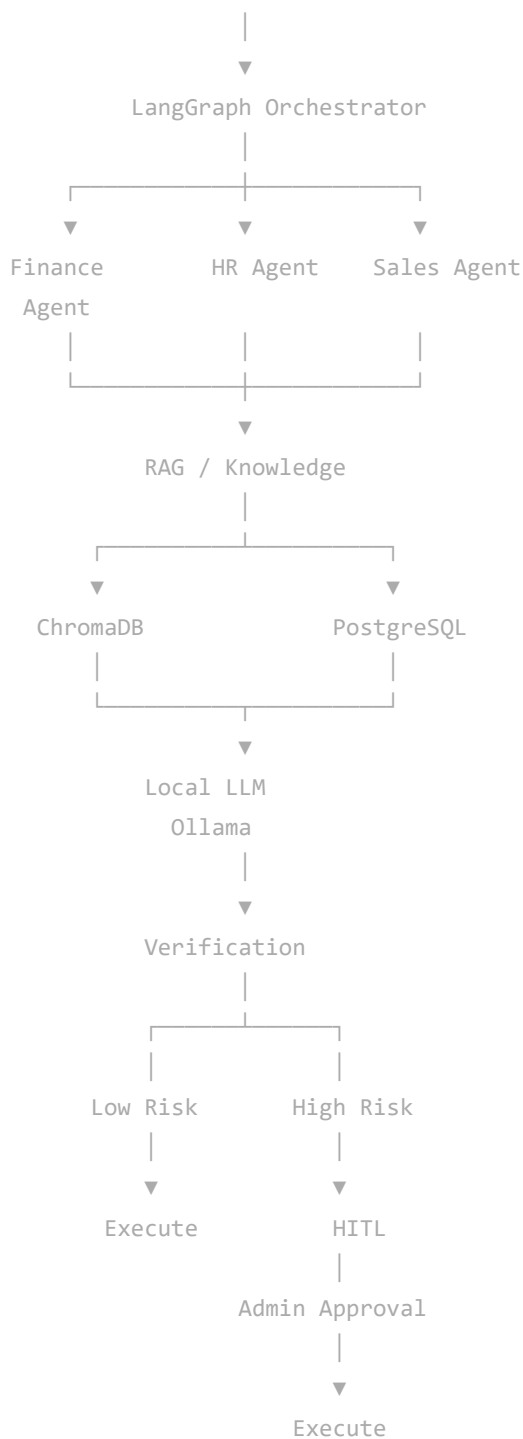
One important distinction: some tools are free/open-source, but an LLM API can still charge based on usage. Since you want **zero paid software**, I'd avoid commercial API dependency altogether.

Recommended 100% free stack

| Component           | Use this                             | Cost             |
|---------------------|--------------------------------------|------------------|
| Frontend            | React + Vite                         | Free             |
| Backend             | Python + FastAPI                     | Free             |
| Agent orchestration | LangGraph                            | Free/open-source |
| LLM                 | Ollama + local model                 | Free             |
| Models              | Llama / Qwen / Gemma via Ollama      | Free             |
| RAG                 | LangChain                            | Free/open-source |
| Vector DB           | ChromaDB                             | Free/open-source |
| Database            | PostgreSQL                           | Free/open-source |
| Embeddings          | sentence-transformers                | Free/local       |
| Authentication      | JWT                                  | Free             |
| HITL                | Build it yourself in React + FastAPI | Free             |
| API communication   | REST/FastAPI                         | Free             |
| Monitoring          | Python logging + custom dashboard    | Free             |
| Development         | VS Code                              | Free             |
| Version control     | Git + GitHub Free                    | Free             |

The architecture I'd recommend for you





## Why I recommend Ollama

Instead of:

LangGraph → OpenAI/Gemini API → pay for tokens

you can do:

**LangGraph → Ollama → local LLM**

The model runs on **your own computer**, so there is no API bill.

You can experiment with models such as:

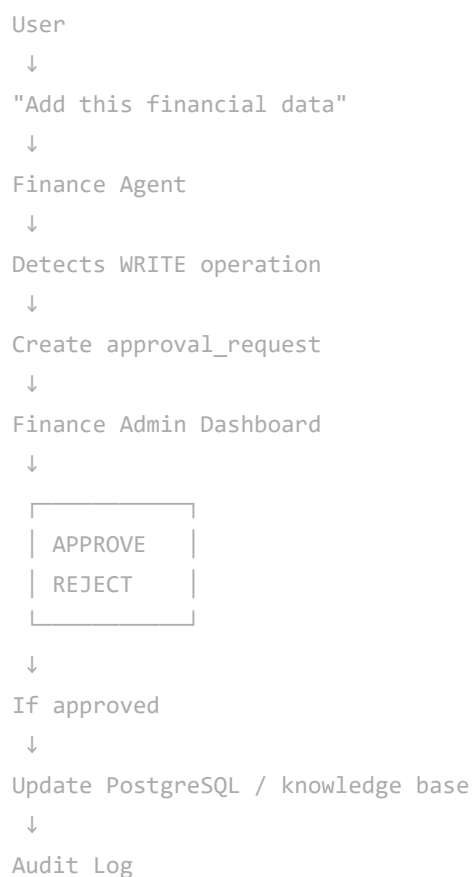
- Qwen
- Llama
- Gemma
- Mistral

depending on your hardware.

## For your HITL feature

You don't need LangSmith, paid workflow software, or another service.

Build a simple approval system yourself:



You can implement the approval request with a simple PostgreSQL table:

approval\_requests

id  
requester  
agent  
action  
data  
risk\_level  
status  
created\_at

approved\_by  
approved\_at

That is actually **better for your case study prototype**, because you can demonstrate that you designed the governance mechanism yourself.

### One thing I'd change from my previous stack

I previously mentioned **LangSmith** for monitoring.

Since you want **zero paid dependencies**, let's remove **LangSmith** completely.

Use:

**Python logging + PostgreSQL + your own React monitoring page.**

So your final stack becomes:

**React + Vite + FastAPI + LangGraph + Ollama + LangChain + ChromaDB +  
PostgreSQL + Sentence Transformers + JWT + Git/GitHub**

All can be used without paying for software.

For the rest of Phase 4, I'll keep this constraint: no paid software, no paid APIs, and preferably no service that requires a credit card.



ChatGPT can make mistakes. Check important info